Итог

Статус: Не готово к сдаче.

По структуре и коду работа содержательная: предусмотрены анализ Vegetable Image Dataset, аугментации, собственная CNN, transfer learning на ResNet18, подбор гиперпараметров по validation, test-оценка, confusion matrix и анализ ошибок. Но фактическое выполнение оборвалось на скачивании датасета: ячейка загрузки завершилась KeyboardInterrupt, а 14 последующих code cells не запускались. Поэтому обучение, реальные метрики, графики, classification report и примеры ошибок не подтверждены.

Отдельное ТЗ к практической №1 не приложено, поэтому аудит опирается на требования, прямо заявленные в notebook, и на минимальные требования воспроизводимого ML-отчёта. После полного GPU-запуска, сохранения outputs и добавления финального вывода работа, вероятно, будет близка к сдаваемой.

Матрица соответствия

№	Источник	Требование	Статус	Доказательство	Замечание
1	Notebook	Оформить работу как Jupyter Notebook с пояснениями Markdown.	выполнено	30 ячеек: 13 Markdown и 17 code.	Структура последовательная.
2	Цель	Решить многоклассовую классификацию овощей.	выполнено	Код рассчитан на 15 классов.	Постановка сформулирована.
3	Данные	Использовать Vegetable Image Dataset: 21 000 изображений, 15 классов.	частично	Описание есть; kagglehub download начат.	Загрузка не завершена.
4	Данные	Использовать train / validation / test.	частично	Код ожидает три исходных каталога.	Фактические размеры не подтверждены.
5	Данные	Проверить распределение классов.	частично	Ячейки 9-10 считают классы и строят bar chart.	Outputs отсутствуют.
6	Данные	Визуализировать исходные изображения.	частично	Ячейка 15 строит сетку примеров.	Вывод не сохранён.
7	Предобработка	Привести изображения к единому размеру.	выполнено	RandomResizedCrop / Resize + CenterCrop до 96.	Код реализован.
8	Предобработка	Использовать аугментации только для train.	выполнено	Flip, rotation, ColorJitter только в train.	Val/test детерминированы.
9	Предобработка	Нормализовать изображения.	выполнено	ImageNet mean/std.	Подходит для pretrained ResNet.
10	Данные	Создать DataLoader для train/val/test.	выполнено	Три DataLoader, shuffle только train.	Код реализован.
11	Воспроизводимость	Зафиксировать seed.	выполнено	SEED=42 для random, NumPy, PyTorch, loader.	Детерминированность предусмотрена.
12	Модель	Реализовать собственную CNN.	выполнено	4 conv-блока, BN, pooling, dropout.	Shape-тест прошёл.
13	Обучение	Реализовать общий train/validation loop.	выполнено	run_epoch + train_model.	Реальный run отсутствует.
14	Метрики	Использовать accuracy и macro-F1.	выполнено	Метрики считаются на train/val/test.	Выбор разумный.
15	Подбор	Подбирать hyperparameters по validation.	частично	По 2 trial для CustomCNN и ResNet.	Не выполнено; бюджет мал.

№	Источник	Требование	Статус	Доказательство	Замечание
16	Подбор	Не использовать test для выбора модели.	выполнено	Выбор по validation macro-F1.	Test оставлен на финал.
17	Custom CNN	Обучить выбранную CNN на полном train.	частично	Ячейка 22 реализует обучение.	Training output отсутствует.
18	Transfer	Использовать pretrained ResNet18.	выполнено	ResNet18_Weights.DEFAULT.	Transfer learning реализован.
19	Transfer	Сначала обучить только новую head.	частично	Backbone freeze, обучение fc.	Результаты отсутствуют.
20	Transfer	Выполнить fine-tuning части backbone.	частично	Размораживается layer4.	Результаты отсутствуют.
21	Checkpoint	Выбирать лучший checkpoint по validation macro-F1.	выполнено	best_state + выбор head/fine-tune.	Логика корректная.
22	Оценка	Оценить test после завершения настройки.	частично	Ячейка 27 вызывает predict.	Test metrics не получены.
23	Сравнение	Сравнить CustomCNN и ResNet18.	частично	comparison_df включает metrics, time, params.	Таблица не выполнена.
24	Визуализация	Построить confusion matrices.	частично	Ячейка 28 строит две матрицы.	Outputs отсутствуют.
25	Анализ	Показать classification report по классам.	частично	Ячейка 29 формирует два отчёта.	Значений нет.
26	Анализ	Показать реальные ошибки модели.	частично	Ячейка 30 выбирает ошибки ResNet18.	Не выполнено.
27	Артефакты	Сохранять checkpoints и графики.	частично	torch.save / plt.savefig в ARTIFACT_DIR.	Файлы не подтверждены.
28	Отчёт	Сформулировать финальный вывод.	не выполнено	После error analysis секции «Вывод» нет.	Отчёт заканчивается кодом.
29	Воспроизводимость	Run All должен завершаться без ошибки.	не выполнено	Ячейка 8: KeyboardInterrupt на 534 MB download.	Главный runtime-блокер.
30	Качество	Не хранить секреты.	выполнено	Статический поиск ничего не выявил.	Секретов не найдено.
31	Качество	Предоставить доказательства фактического обучения.	не выполнено	Выполнены 3 из 17 code cells.	Нет метрик/графиков/моделей.
32	Воспроизводимость	Запускаться в чистой среде с понятными зависимостями/путями.	частично	kagglehub без install cell; /content при Windows run.	Среда не унифицирована.
33	Методология	Не допустить утечку test в настройку.	выполнено	Test не используется до финальной оценки.	Методологически корректно.
34	Сравнение	Сопоставить параметры и время обучения.	частично	Код считает params и seconds.	Реальные числа отсутствуют.

Найденные проблемы

AUD-001 - Critical

Источник: Качество / доказательства выполнения

Описание: Скачивание датасета было прервано, после чего обучение и оценка не выполнялись. Нельзя подтвердить CustomCNN/ResNet18, test-метрики, графики и ошибки.

Как воспроизвести или проверить: Проверить ячейку 8, execution_count и outputs последующих code cells.

Ожидаемый и фактический результат: Ожидается полный Run All с результатами. Фактически cell 8 содержит KeyboardInterrupt; выполнены только 3 из 17 code cells.

Рекомендуемое исправление: Запустить notebook в Colab с GPU и стабильным интернетом, выполнить сверху вниз и сохранить outputs.

AUD-002 - Major

Источник: Отчёт / итоговый анализ

Описание: Отдельного раздела «Вывод» нет: notebook заканчивается ячейкой визуализации ошибок.

Как воспроизвести или проверить: Проверить последние ячейки 26-30.

Ожидаемый и фактический результат: Ожидается финальное сравнение моделей, реальные accuracy/macro-F1, сильные/слабые классы и ограничения. Фактически заключение отсутствует.

Рекомендуемое исправление: Добавить программно формируемый вывод после полного запуска, используя comparison_df и class reports.

AUD-003 - Major

Источник: Качество эксперимента / hyperparameter tuning

Описание: Экспериментальный бюджет слишком мал: SEARCH_EPOCHS=1, CUSTOM_EPOCHS=4, TRANSFER_HEAD_EPOCHS=2, FINETUNE_EPOCHS=1. Один epoch поиска может ранжировать настройки по шуму.

Как воспроизвести или проверить: Проверить ячейку параметров эксперимента и циклы trial.

Ожидаемый и фактический результат: Ожидается достаточно длинный run для стабилизации validation-метрик. Фактически поиск каждой конфигурации длится 1 epoch.

Рекомендуемое исправление: Увеличить эпохи или добавить early stopping; сохранить кривые и показать достижение плато.

AUD-004 - Major

Источник: Анализ ошибок / согласованность выбора модели

Описание: Error analysis жёстко использует ResNet18 независимо от результата сравнения. Если CustomCNN окажется лучше, финальный анализ будет несогласован с выбранной моделью.

Как воспроизвести или проверить: Проверить ячейку 30: y_true=transfer_true, y_pred=transfer_pred.

Ожидаемый и фактический результат: Ожидается анализ выбранной по validation модели либо явное объяснение, почему анализируется только ResNet18.

Рекомендуемое исправление: Выбирать модель по validation macro-F1 или показать ошибки обеих моделей.

AUD-005 - Minor

Источник: Transfer learning / preprocessing

Описание: ResNet18 получает 96×96 вместо стандартного ImageNet масштаба около 224×224. Это допустимо технически, но может терять детали и снижать пользу pretrained признаков.

Как воспроизвести или проверить: Проверить IMAGE_SIZE=96 и eval_transform.

Ожидаемый и фактический результат: Ожидается обоснованный компромисс скорость/качество; отдельного сравнения resolution нет.

Рекомендуемое исправление: Либо использовать 224×224, либо сравнить 96 и 224 на validation и обосновать выбор.

AUD-006 - Minor

Источник: Воспроизводимость / пути

Описание: ARTIFACT_DIR задан как /content/lab1_artifacts, однако сохранённый запуск выполнен локально на Windows и CPU.

Как воспроизвести или проверить: Сопоставить ячейку 6 с output kagglehub C:\Users\... и device=cpu.

Ожидаемый и фактический результат: Ожидается единая runtime-aware логика путей. Фактически Colab-путь используется и в local run.

Рекомендуемое исправление: Определять IN_COLAB; локально писать в относительный lab1_artifacts/.

AUD-007 - Minor

Источник: Воспроизводимость / зависимости

Описание: Notebook импортирует kagglehub, но не содержит install cell или явной инструкции установки.

Как воспроизвести или проверить: Найти import kagglehub и проверить наличие %pip install / ссылки на requirements.

Ожидаемый и фактический результат: Ожидается запуск в чистой среде; в notebook установка зависимости не описана.

Рекомендуемое исправление: Добавить компактную install cell либо команду pip install -r requirements.txt.

Непроверяемые пункты

- Полная загрузка Vegetable Image Dataset и реальные 15 000 / 3 000 / 3 000 изображений.
- Hyperparameter search, обучение CustomCNN и ResNet18, checkpoints.
- Реальные accuracy/precision/recall/macro-F1 на test.
- Confusion matrices, class reports и примеры ошибок.
- Полный Restart runtime -> Run all без traceback.

Лишняя реализация

- Выбор checkpoint по validation macro-F1 вместо test.
- ReduceLROnPlateau в общем train loop.
- Подсчёт общего и trainable числа параметров.
- Стратифицированный subset для быстрого поиска.
- Двухэтапный transfer learning: frozen head -> fine-tuning layer4.
- Фильтрация почти дублирующихся изображений при выборе ошибок.
- Сохранение checkpoints и графиков в отдельный каталог.

Эти элементы не являются лишними в отрицательном смысле: они усиливают работу и демонстрируют аккуратную ML-инженерию, если итоговый notebook действительно выполнен.

Результаты проверок

- Файл: 59 624 байта; SHA256 18DE4F03BD7577E56C005DFDABC40CED3FB5EAFA9E97F90DE8CF1561B946AA78; nbformat 4.5.
- nbformat.validate: успешно.
- Состав: 30 ячеек - 13 Markdown и 17 code; execution_count есть у 3 code cells; всего 5 outputs.
- Runtime error: cell 8 содержит KeyboardInterrupt во время kagglehub.dataset_download; download остановлен примерно на 2 MB из 534 MB.
- AST: все 17 code cells синтаксически корректны.
- Секреты: очевидные API keys, tokens, passwords не обнаружены.
- CustomCNN smoke test: [4,3,96,96] -> [4,15], finite=True; 317 935 параметров.
- denormalize: синтетическая round-trip проверка прошла, max error около 6e-8.
- metric_summary: synthetic test корректно выдаёт accuracy/precision/recall/macro-F1.
- Полное обучение: в аудите не запускалось; notebook сам не содержит подтверждённых результатов после загрузки.

Подготовка к защите

Сценарий демонстрации работы по шагам

1. Открыть notebook в Colab, включить GPU и показать DEVICE=cuda.
2. Проверить зависимости и полностью скачать Vegetable Image Dataset.
3. Показать DATA_ROOT и реальные размеры train/validation/test.
4. Показать распределение 15 классов и исходные изображения.
5. Объяснить train/eval transforms и ImageNet normalization.
6. Показать CustomCNN, число параметров и hyperparameter trials.
7. Показать кривые loss / accuracy / macro-F1 CustomCNN.
8. Показать ResNet18: frozen backbone -> head training -> fine-tuning layer4.
9. Показать validation результаты и выбранный checkpoint.
10. Показать comparison_df на test.
11. Показать confusion matrices и classification reports.
12. Показать 4-6 реальных ошибок с truth/prediction/confidence.
13. Закончить финальным выводом по фактическим числам.

Обязательные экраны, таблицы и артефакты

- GPU/CUDA и версии Python/PyTorch/torchvision.
- Размеры train/validation/test и class distribution.
- Примеры изображений после denormalize.
- Таблицы hyperparameter search обеих моделей.
- Графики train/validation loss, accuracy, macro-F1.
- Trainable parameters ResNet18 до/после разморозки layer4.
- Итоговая comparison_df.
- Confusion matrices + classification reports.
- Примеры ошибок + .pth checkpoints.
- Финальный вывод.

10 вероятных вопросов преподавателя и краткие ответы

1. Почему Vegetable Image Dataset? 15 классов и готовое train/validation/test разбиение подходят для сравнения CNN и transfer learning.
2. Зачем validation? Для hyperparameter tuning и выбора checkpoint; test должен оставаться независимым.
3. Почему macro-F1? Он одинаково учитывает каждый класс и дополняет accuracy.
4. Что делает BatchNorm? Стабилизирует распределение промежуточных активаций и обучение.
5. Зачем dropout? Снижает переобучение классификатора.
6. Что такое transfer learning? Использование признаков ResNet18, заранее обученных на ImageNet, с адаптацией head и части backbone.
7. Почему backbone сначала frozen? Чтобы сначала адаптировать head и не разрушить pretrained признаки крупными обновлениями.
8. Почему test только в конце? Чтобы не подгонять решения под итоговую выборку.
9. Что показывает confusion matrix? Какие реальные классы систематически путаются между собой.
10. Какие фактические метрики получены? В приложенном состоянии их нет: выполнение прервано на загрузке, поэтому до защиты нужно сделать полный run.

Что обязательно исправить или подготовить до сдачи

- Выполнить notebook полностью и сохранить outputs.
- Получить реальные validation/test метрики обеих моделей.
- Сохранить curves, confusion matrices, class reports, error examples.
- Добавить финальный раздел «Вывод».
- Увеличить training/search budget или обосновать короткий run графиками.
- Согласовать error analysis с моделью, выбранной по validation.
- Проверить пути артефактов и чистый запуск зависимостей.

Решение по приёмке

Обязательно исправить до сдачи

- Завершить загрузку датасета и выполнить весь Notebook.
- Предъявить реальные validation/test метрики CustomCNN и ResNet18.
- Сохранить графики обучения, confusion matrices, classification reports и примеры ошибок.
- Добавить итоговый вывод на основе фактических чисел.
- Подтвердить, что Run All проходит без KeyboardInterrupt и других ошибок.

Желательно исправить

- Увеличить число эпох в search/final training или использовать early stopping.
- Сделать модель для error analysis динамической по validation macro-F1.
- Проверить ResNet18 на 224×224 либо обосновать 96×96.
- Сделать runtime-aware ARTIFACT_DIR.
- Явно описать установку kagglehub / requirements.

Можно объяснить как ограничение работы

- Только 2 hyperparameter конфигурации на модель из-за ограниченного GPU-времени.
- 96×96 ради скорости, если validation подтверждает приемлемое качество.
- Короткий fine-tuning, если validation-кривые стабилизировались.
- CPU как запасной режим; финальный эксперимент лучше на GPU.

Требуется уточнение у преподавателя

- Какой training budget считается достаточным?
- Нужно ли прикладывать .pth checkpoints отдельно?
- Достаточно ли accuracy + macro-F1?
- Нужно ли сравнивать inference speed?
- Допустим ли вход 96×96 для ResNet18 или ожидается 224×224?

Финальный вердикт	Не готово к сдаче в текущем сохранённом состоянии. Кодовая основа хорошая, но экспериментально подтверждённых результатов нет из-за прерванной загрузки датасета. После полного GPU-запуска, сохранения outputs и добавления вывода работу можно довести до сдаваемого состояния без переписывания архитектуры с нуля.